

CP468-A, Spring 25

Final Project: Multi-Agent Decision Making in Games

Dr. Ahmed Ibrahim

Due: July 29th, 2025

Completed By:

Ahmad Wali	wali6947@mylaurier.ca	169036947
William Ni	nixx9887@mylaurier.ca	169019887
Baajdeep Boparai	bopa6411@mylaurier.ca	169026411
Karmanjot Baidwan	baid7691@mylaurier.ca	169037691
Adrian Taccogna	tacc9851@mylaurier.ca	169029851
Dillon Quick	quic1522@mylaurier.ca	169031522
Amirhossein Ahmadinoori	ahma2860@mylaurier.ca	169042860
Devang Malhotra	malh2226@mylaurier.ca	169022226
Matthew Zoretich	zore0554@mylaurier.ca	169020554

Section 1: Adversarial Game Components

Design:

We implemented Connect Four as our adversarial game - with a stochastic twist - as a perfect information, two-player game. Our game supports Human vs AI Agent, and AI vs AI. Our game enforces legal moves and detects win conditions. We built our Flask backend in Python, which manages the game state and game logic, while ReactJS was used to display the front-end.

A **dropRandomPiece()** function introduces stochastic blocking pieces with a 25% probability after every few turns. The ExpectiMiniMax algorithm handles this probabilistic element, while Minimax and Alpha-Beta assume deterministic play.

A Flask backend exposes the API endpoints that receive the current board state from the React frontend. It also queries the Gemini API for suggested moves by passing a formatted board representation. It returns the AI's move to the frontend for rendering. Gemini API acts as an external LLM agent for decision-making and can be benchmarked against the internal algorithms

The backend uses Python and Flask. The backend hosts the game logic and does AI decision-making. It also provides the API endpoints. The frontend is ReactJS. This displays the board dynamically, sends player moves to Flask, and receives AI moves asynchronously via fetch calls. The decoupled architecture allows swapping different AI agents without changing the UI.

While building the full stack, we also implemented a makeshift API using a .txt file to test our components. During our development, we wrote the board state to a text file, which allows for faster iteration and debugging. Flask handles the backend logic, whereas ReactJS will send moves and update using calls.

State Representation:

State space for normal version: $O(3^{(n*m)})$.

- Where n, m is the height and width of the board, 3 represents the number of states of each piece (taken by red, taken by yellow, not taken)

State space for stochastic version: $O(3^{(n*m)})$.

- Similar to above, n and m represent the same thing. Here, we technically have a 4th state (random node's) however, due to their non-common probability of dropping, it's easier to abstract their involvement and keep it at 3 possible states.

Multi-Agent Setup (Gemini Integration)

The implementation of Google's Gemini began with generating an API key from Google AI Studio. Afterwards, we added this API key to our Python backend file, since storing it in the frontend would lead to this key being exposed for other users to see. Following this, we created a **gemini_prompt** function within our backend which takes the current state of the 2D board, the total number of

rows, and the total number of columns as parameters. The next stop involves creating a detailed prompt and generating a response, for which we made use of the Gemini 2.5 Pro model.

```
def gemini_prompt(board, rows, cols):
    client = genai.Client(api_key=API_KEY)
    prompt = f"You are playing a connect 4 game with an opponent. The current board looks like {board}. There are {rows} rows and {cols} columns. 0 represents an empty spot, 1 represents a spot taken by the opponent, and -1 represents the moves you have played. 9 represents a blocker piece (neither you or the opponent can place a piece there). Which column should you drop your piece in to make the best move? Only provide the column number (starting from 1 to {cols}) with no explanation."
    response = client.models.generate_content(
        model="gemini-2.5-pro",
        contents=prompt
    )
    return response.text
```

Section 2: Data Structure & Algorithm Analysis

Evaluation Heuristic:

The Evaluation Heuristic implements a 4 * 4 sliding window algorithm which scans all possible “windows” on our n * m board. The heuristic applies value to:

1. Dominating Centre 4 squares
2. Having consecutive coloured pieces lined up

This is the foundation of our search algorithm’s to asses non-terminal game states, and is part of all of the other algorithms in providing a heuristic score. The pseudo code is as follows:

```
FUNCTION evaluate_window(window):
    count_player = count of player pieces in window
    count_ai = count of AI pieces in window
    count_empty = count of empty spaces in window

    IF both players have pieces in window THEN
        RETURN 0 // Blocked window, no potential

    score = 0
    IF only player pieces present THEN
        IF count_player == 4 THEN score += WIN_SCORE
        ELSE IF count_player == 3 AND count_empty == 1 THEN score += 5
```

```

    ELSE IF count_player == 2 AND count_empty == 2 THEN score += 2

    IF only AI pieces present THEN
        IF count_ai == 4 THEN score -= WIN_SCORE
        ELSE IF count_ai == 3 AND count_empty == 1 THEN score -= 4
        ELSE IF count_ai == 2 AND count_empty == 2 THEN score -= 2

    RETURN score

FUNCTION evaluate_board(board):
    IF player has won THEN RETURN WIN_SCORE
    IF AI has won THEN RETURN -WIN_SCORE
    IF board is full THEN RETURN 0

    total_score = 0

    // Center column preference (Connect 4 specific)
    center_column = board middle column
    total_score += 3 * (player_pieces_in_center - ai_pieces_in_center)

    // Evaluate all possible 4-piece windows
    FOR each horizontal window DO
        total_score += evaluate_window(window)
    FOR each vertical window DO
        total_score += evaluate_window(window)
    FOR each diagonal window DO
        total_score += evaluate_window(window)

    RETURN total_score

```

MiniMax:

MiniMax works by assuming that both players are playing optimally. It recursively explores the game tree, and aims to pick the choice that helps the maximizing player maximize their score, which would in turn be the choice that minimizes the score for the opponent. It calls upon the evaluation heuristic earlier to get scores of each game state to understand which one is the best pick for itself. In this specific context of Connect 4, it will prioritize centre columns, it's max computational depth is 4-8 moves, and the terminal states also include draw conditions.

The pseudocode implementation of MiniMax:

```

FUNCTION minimax_value(board, depth, maximizing_player):
    node_count += 1 // Performance tracking

    IF depth == 0 OR game_over(board) THEN
        RETURN evaluate_board(board)

```

```

IF maximizing_player THEN
    max_eval = -infinity
    FOR each valid column DO
        make_move(board, column, PLAYER_PIECE)
        eval = minimax_value(board, depth-1, FALSE)
        undo_move(board, column)
        max_eval = max(max_eval, eval)
    RETURN max_eval
ELSE
    min_eval = +infinity
    FOR each valid column DO
        make_move(board, column, AI_PIECE)
        eval = minimax_value(board, depth-1, TRUE)
        undo_move(board, column)
        min_eval = min(min_eval, eval)
    RETURN min_eval

```

```

FUNCTION minimax_move(board, depth):
    // Determine if current player is maximizing based on move count
    maximizing = (total_pieces_on_board % 2 == 0)
    best_value = maximizing ? -infinity : +infinity
    best_column = null

    FOR each valid column DO
        make_move(board, column, current_piece)
        value = minimax_value(board, depth-1, !maximizing)
        undo_move(board, column)

        IF (maximizing AND value > best_value) OR
            (!maximizing AND value < best_value) THEN
            best_value = value
            best_column = column

    RETURN best_column

```

Alpha-Beta Pruning:

Alpha-Beta pruning is an extension of MiniMax, which works by cutting off branches of our search tree that we know will not affect our final decision. In this case, Alpha represents the best value the maximizing player can guarantee and Beta is the best value the minimizing player can guarantee. By “pruning” these branches of our tree, we optimize MiniMax to be even faster.

The pseudocode implementation is as follows:

```

FUNCTION alpha_beta_value(board, depth, alpha, beta, maximizing_player):
    node_count += 1

```

```

IF depth == 0 OR game_over(board) THEN
    RETURN evaluate_board(board)

IF maximizing_player THEN
    value = -infinity
    FOR each valid column DO
        make_move(board, column, PLAYER_PIECE)
        value = max(value, alpha_beta_value(board, depth-1, alpha, beta, FALSE))
        undo_move(board, column)

        alpha = max(alpha, value)
        IF alpha >= beta THEN
            BREAK // Beta cutoff - prune remaining branches
    RETURN value
ELSE
    value = +infinity
    FOR each valid column DO
        make_move(board, column, AI_PIECE)
        value = min(value, alpha_beta_value(board, depth-1, alpha, beta, TRUE))
        undo_move(board, column)

        beta = min(beta, value)
        IF beta <= alpha THEN
            BREAK // Alpha cutoff - prune remaining branches
    RETURN value

FUNCTION alpha_beta_move(board, depth):
    maximizing = (total_pieces_on_board % 2 == 0)
    best_value = maximizing ? -infinity : +infinity
    best_column = null
    alpha = -infinity
    beta = +infinity

    FOR each valid column DO
        make_move(board, column, current_piece)
        value = alpha_beta_value(board, depth-1, alpha, beta, !maximizing)
        undo_move(board, column)

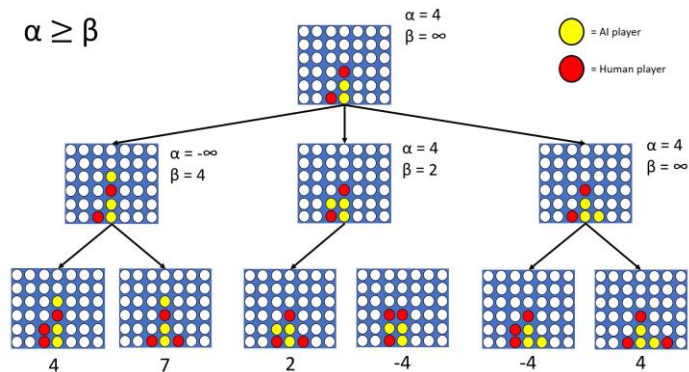
        IF (maximizing AND value > best_value) OR
            (!maximizing AND value < best_value) THEN
            best_value = value
            best_column = column

    // Update alpha/beta bounds for further pruning
    IF maximizing THEN alpha = max(alpha, best_value)
    ELSE beta = min(beta, best_value)

    IF beta <= alpha THEN BREAK // Prune remaining moves

```

RETURN best_column



Note: This is not a hand made visual, please see references 5.a for citations.

ExpectiMiniMax:

ExpectiMiniMax is an extension of MiniMax, which handles stochastic elements to our game. In our context, we have 2 versions of our game, one that includes random pieces being dropped and one that doesn't. In the case that we are playing traditional connect 4, we assume that the other player is not striving for optimal play. ExpectiMiniMax handles this by assuming uniform probability. A sample computation is provided below:

Expected Value = $\sum (P(\text{move}_i) \times \text{Value}(\text{resulting_state}_i))$

If 7 valid moves with uniform probability:

$P(\text{each move}) = 1/7 \approx 0.143$

Expected Value = $0.143 \times V_1 + 0.143 \times V_2 + \dots + 0.143 \times V_7$

In this case, we implement ExpectiMiniMax as follows:

```
FUNCTION expectiminimax_value(board, depth, node_type, move_probabilities):
    node_count += 1

    IF depth == 0 OR game_over(board) THEN
        RETURN evaluate_board(board)

    valid_moves = get_valid_columns(board)

    IF node_type == "max" THEN
        max_value = -infinity
        FOR each valid column DO
            make_move(board, column, PLAYER_PIECE)
            value = expectiminimax_value(board, depth-1, "chance", move_probabilities)
```

```

    undo_move(board, column)
    max_value = max(max_value, value)
    RETURN max_value

ELSE IF node_type == "min" THEN
    min_value = +infinity
    FOR each valid column DO
        make_move(board, column, AI_PIECE)
        value = expectiminimax_value(board, depth-1, "chance", move_probabilities)
        undo_move(board, column)
        min_value = min(min_value, value)
    RETURN min_value

ELSE // chance node
    IF move_probabilities == null THEN
        // Uniform probability distribution
        probability_per_move = 1.0 / length(valid_moves)
        FOR each valid column DO
            move_probabilities[column] = probability_per_move

    expected_value = 0.0
    current_player = determine_current_player(board)
    next_node_type = current_player == PLAYER ? "min" : "max"

    FOR each valid column DO
        probability = move_probabilities[column]
        make_move(board, column, current_player)
        value = expectiminimax_value(board, depth-1, next_node_type, move_probabilities)
        undo_move(board, column)
        expected_value += probability * value

    RETURN expected_value

```

Assuming we are playing with dropping random nodes, we account for them by multiplying the probability of them NOT dropping (75%) and use that to weigh the heuristic value of each column (similar to above)

A sample computation is as follows:

$$\text{Expected Value} = P(\text{no_drop}) \times \text{Value}(\text{no_drop_state}) + \sum (P(\text{drop_in_col_i}) \times \text{Value}(\text{state_with_drop_in_col_i}))$$

Where:

$$P(\text{no_drop}) = 0.75$$

$$P(\text{drop_in_col_i}) = 0.25 / \text{number_of_valid_columns}$$

If 7 valid columns:

$P(\text{drop in any specific column}) = 0.25/7 \approx 0.036$
--

The pseudo-code implementation of ExpectiMiniMax, for when we play with random piece drops:

```
FUNCTION expectiminimax_random_value(board, depth, node_type, move_probabilities,
random_drop_prob):
    node_count += 1

    IF depth == 0 OR game_over(board) THEN
        RETURN evaluate_board_with_random(board) // Modified evaluation

    // Handle max/min nodes similar to standard expectiminimax
    IF node_type == "max" OR node_type == "min" THEN
        // Similar logic but call expectiminimax_random_with_drop for next level

    ELSE // chance node
        expected_value = 0.0
        current_player = determine_current_player(board)
        next_node_type = current_player == PLAYER ? "min" : "max"

        FOR each valid column DO
            probability = move_probabilities[column]
            make_move(board, column, current_player)
            value = expectiminimax_random_with_drop(board, depth-1, next_node_type,
                move_probabilities, random_drop_prob)
            undo_move(board, column)
            expected_value += probability * value

        RETURN expected_value

FUNCTION expectiminimax_random_with_drop(board, depth, next_node_type,
move_probabilities, random_drop_prob):
    expected_value = 0.0

    // Case 1: No random drop occurs
    no_drop_probability = 1.0 - random_drop_prob
    no_drop_value = expectiminimax_random_value(board, depth, next_node_type,
        move_probabilities, random_drop_prob)
    expected_value += no_drop_probability * no_drop_value

    // Case 2: Random drop occurs
    IF random_drop_prob > 0 THEN
        drop_valid_columns = get_valid_columns_for_drop(board)

        IF drop_valid_columns is not empty THEN
            drop_probability_per_column = random_drop_prob / length(drop_valid_columns)
```

```
FOR each drop_column in drop_valid_columns DO
    make_random_drop(board, drop_column, RANDOM_PIECE)
    drop_value = expectiminimax_random_value(board, depth, next_node_type,
                                              move_probabilities, random_drop_prob)
    undo_random_drop(board, drop_column)
    expected_value += drop_probability_per_column * drop_value
ELSE
    // No valid drop positions, add probability to no-drop case
    expected_value += random_drop_prob * no_drop_value

RETURN expected_value
```

Data Structures:

1. Numpy Arrays

We use Numpy Arrays to help create the board itself. The benefits of this is that it let's us check when the board is empty, create copies and access the count of pieces efficiently with built-in functions. By using a Numpy board we also implement faster array access which speeds up computations. Examples of instance where we use numpy functions are:

Np.count_nonzero(board)	Counts total pieces quickly
Board.count()	Returns the number of occupied spaces in the board
Board.copy()	Let's us create duplicates of the board for move simulation
Board==EMPTY	A check for our terminal states

2. Python Lists (Array based)

We used Lists as a way of creating our sliding window heuristic, valid move's and random choices list. Their implementation helped with heuristic evaluation (a), branching factor control (b) and error handling(c).

a.	In connect4game.py, evaluate_board() function: In connect4game.py evaluate_board() center = list(board[:, NUM_COL//2]) # Convert column to list window = list(board[r, c:c+4]) # Horizontal windows window = [board[r+i][c] for i in range(4)] # Vertical windows
b.	In algorithms.py:

	<pre> valid_moves = [col for col in range(connect4game.NUM_COL) if connect4game.isValidMove(board, col)] drop_valid_cols = [col for col in range(connect4game.NUM_COL) if connect4game.getNextRow(board, col) is not None] </pre>
c.	<pre> In randomConnectFour.py: return random.choice([c for c in range(connect4game.NUM_COL) if connect4game.isValidMove(board, c)]) </pre>

3. Dictionaries

We used Dictionaries help us efficiently test our algorithms(a), store our tournament testing results (b) and help optimize ExpectiMiniMax (c). When optimizing ExpectiMiniMax, we use dictionaries to map each column to its selection probability, this makes it not only easy to implement a default uniform distribution but also allows easier modifications of probabilities per column

a.	<pre> In test1.py: algorithms_dict = { 'Random': random_move, 'Minimax': algorithms.minimax_move, 'AlphaBeta': algorithms.alpha_beta_move, 'ExpectiMinimax': algorithms.expectiminimax_move, 'Gemini': gemini_move, } </pre>
b.	<pre> In test1.py: results = {} results[key] = {'A_wins': 0, 'B_wins': 0, 'Draws': 0} algorithm_stats = {name: {'wins': 0, 'losses': 0, 'draws': 0, 'games': 0}} </pre>
c.	<pre> In expectiminimax algorithms: if move_probabilities is None: prob_per_move = 1.0 / len(valid_moves) move_probabilities = {col: prob_per_move for col in valid_moves} prob = move_probabilities.get(col, 1.0 / len(valid_moves)) </pre>

Section 3: Testing & Evaluation:

Execution Time:

After correcting our Alphabeta logic, the Alphabeta algorithm significantly improved over Minimax by reducing nodes explored by 65.5%, achieving a 4.9 times efficiency increase on a 6x7 board with random piece dropping, and a 5.2 times efficiency increase without random piece dropping.

Execution times scaled better for AlphaBeta across larger board sizes, notably maintaining lower growth rates in execution time compared to Minimax and Expectiminimax.

Node Evaluations:

Both Minimax and Expectiminimax consistently expanded identical numbers of nodes (e.g., 2800 nodes for single moves at depth 4). Alphabeta consistently evaluated significantly fewer nodes, such as 965 nodes on the 6x7 board and 1360 nodes on the 8x9 board compared to 2800 and 7380 nodes respectively for Minimax and Expectiminimax. AlphaBeta achieved the highest efficiency with 10,781 nodes evaluated per second, surpassing Minimax (9,783 nodes/second) and Expectiminimax (10,024 nodes/second).

Success Rate:

Minimax, Alphabeta, and Expectiminimax each secured a 62.5% win rate against other agents, while Random only achieved a 12.5% win rate without Gemini. Additional trials incorporating random piece dropping (25% probability after four turns without Gemini testing) increased the Random algorithm's win rate to 12.9% (+4%) while Expectiminimax's rate dropped to 62.1% (-4%) when we conducted 100 trials per match. With Gemini API Integration, Minimax, AlphaBeta, and Expectiminimax each maintained a consistent win rate of 70%, significantly outperforming Random, which had a win rate of 10%, and the Gemini agent with a win rate of 30%.

Scalability:

AlphaBeta consistently demonstrated lower execution times across board sizes compared to Minimax and Expectiminimax, particularly noticeable in larger board dimensions. Initial results showed the first-moving agent had a notable advantage. Implementing a turn-based system alternating starting players slightly increased variability in outcomes but yielded minimal overall change. A complete game of Connect 4 has approximately $3^{(6 \times 7)}$, or roughly 4.5×10^{12} possible board states. To avoid reaching this upper computational bound, we implemented a depth traversal limit of 4. This strategy ensures that the algorithms remain efficient and feasible, as most computers cannot process that many outcomes.

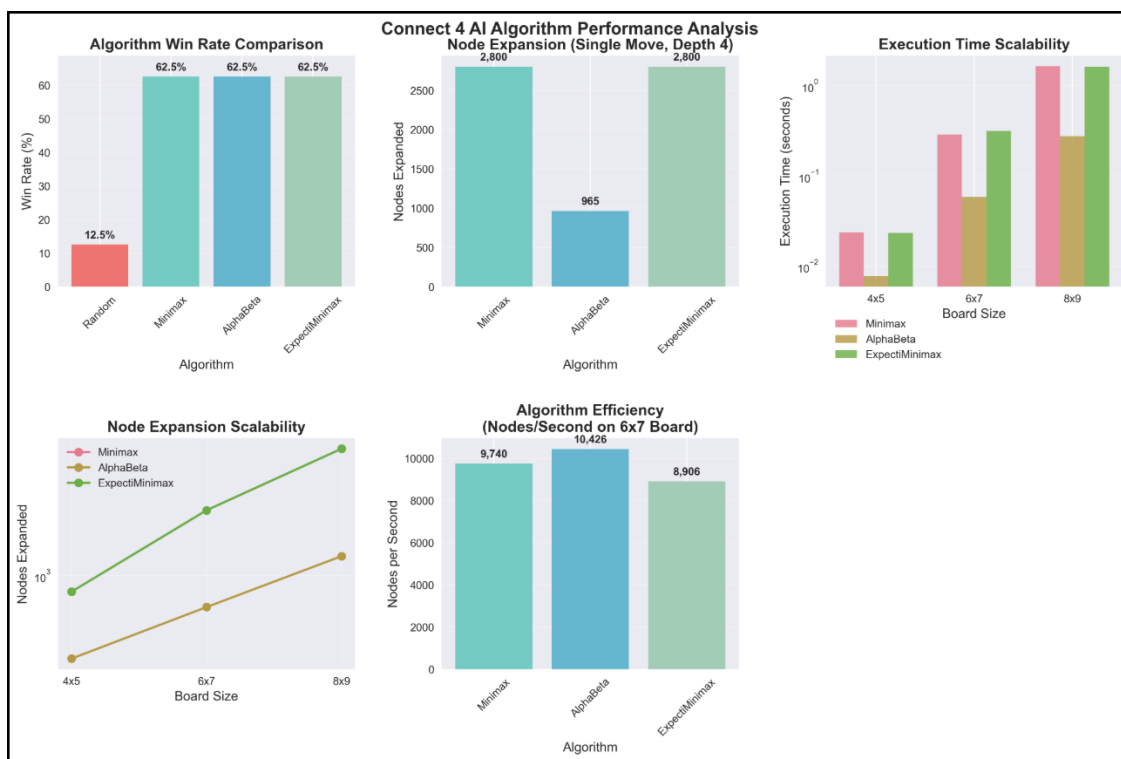
Gemini API Testing:

Gemini testing was constrained by errors 503 and 429, reducing the total number of feasible trials and influencing the scope of results. The program defaults to Gemini, placing its piece in a random position rather than a calculated one. For future implementation, there should be a fallback algorithm that mirrors the AI's logic. Detailed Gemini AI prompts were refined with clearer

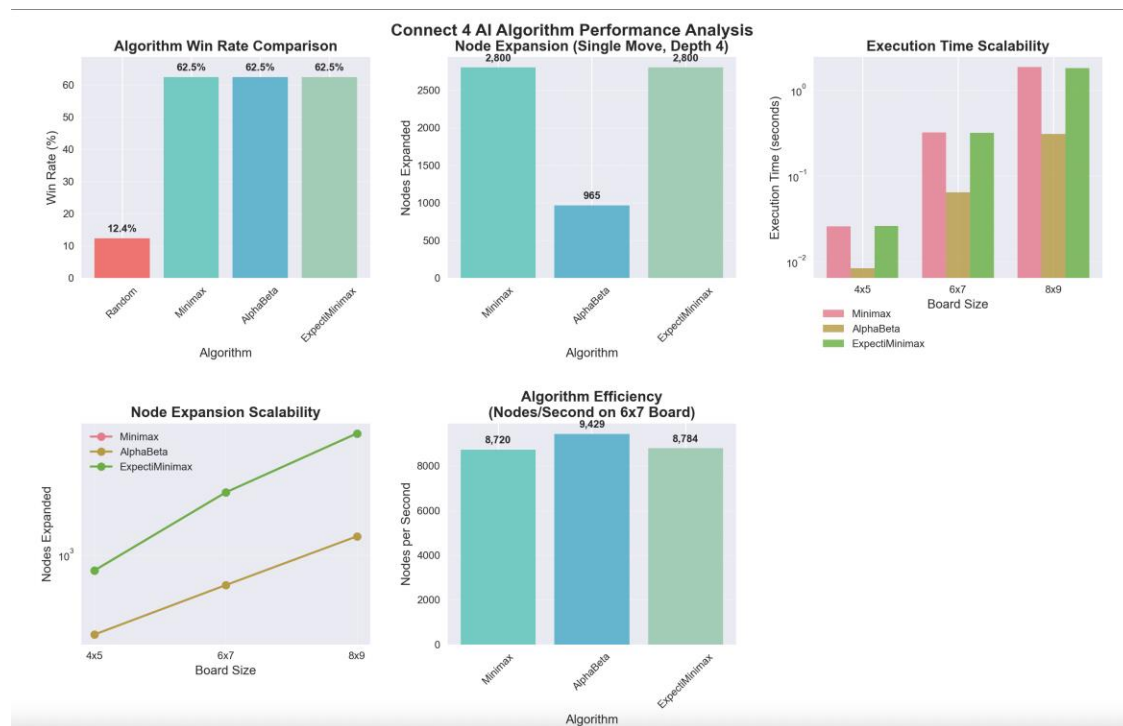
instructions, specifying explicit problem contexts, objectives, and expected outputs to enhance accuracy and clarity of results.

- Example Error Output:
Match 15/25: AlphaBeta vs Gemini
Trial 1/2 -> A (AlphaBeta started)
Trial 2/2 Gemini error: 503 UNAVAILABLE. {'error': {'code': 503, 'message': 'The model is overloaded. Please try again later.', 'status': 'UNAVAILABLE'}}, using random move

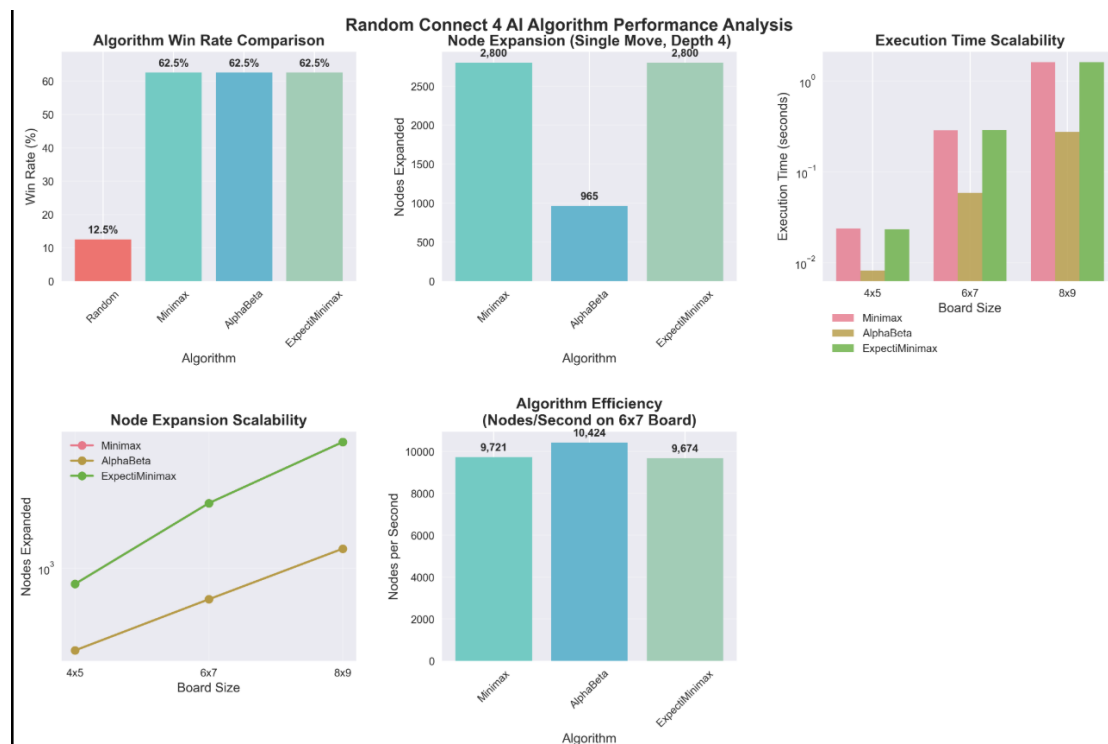
Connect 4 Agent vs Agent (No Gemini): Trials = 3



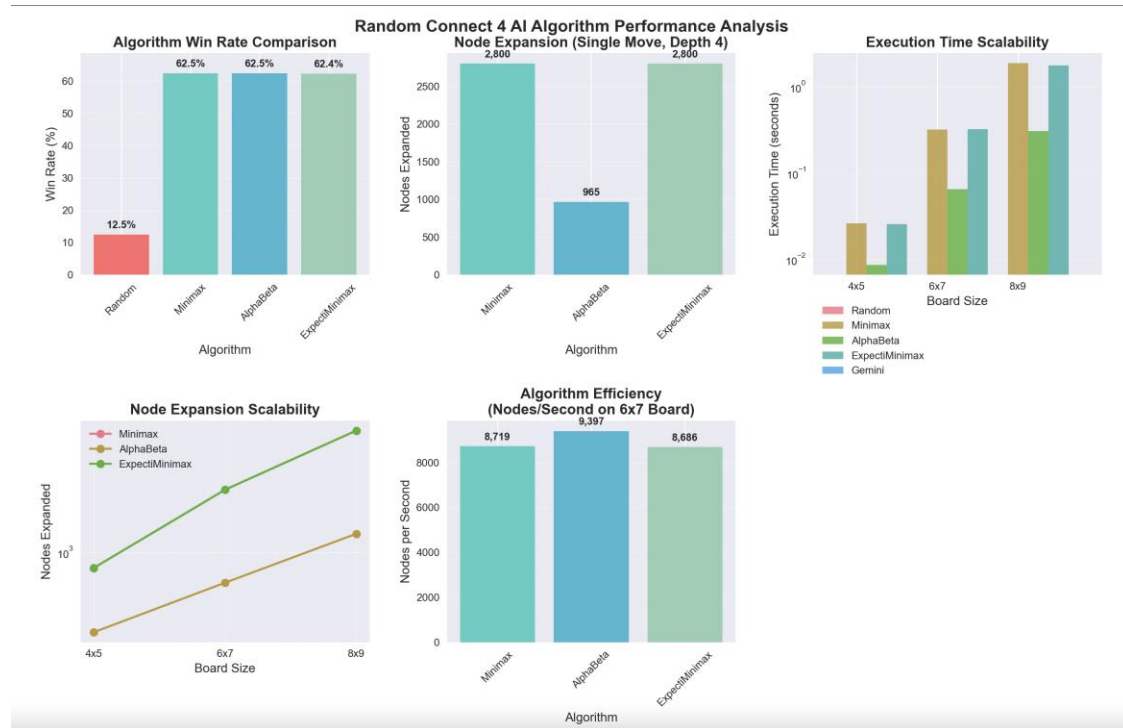
Connect 4 Agent vs Agent (No Gemini): Trials = 100



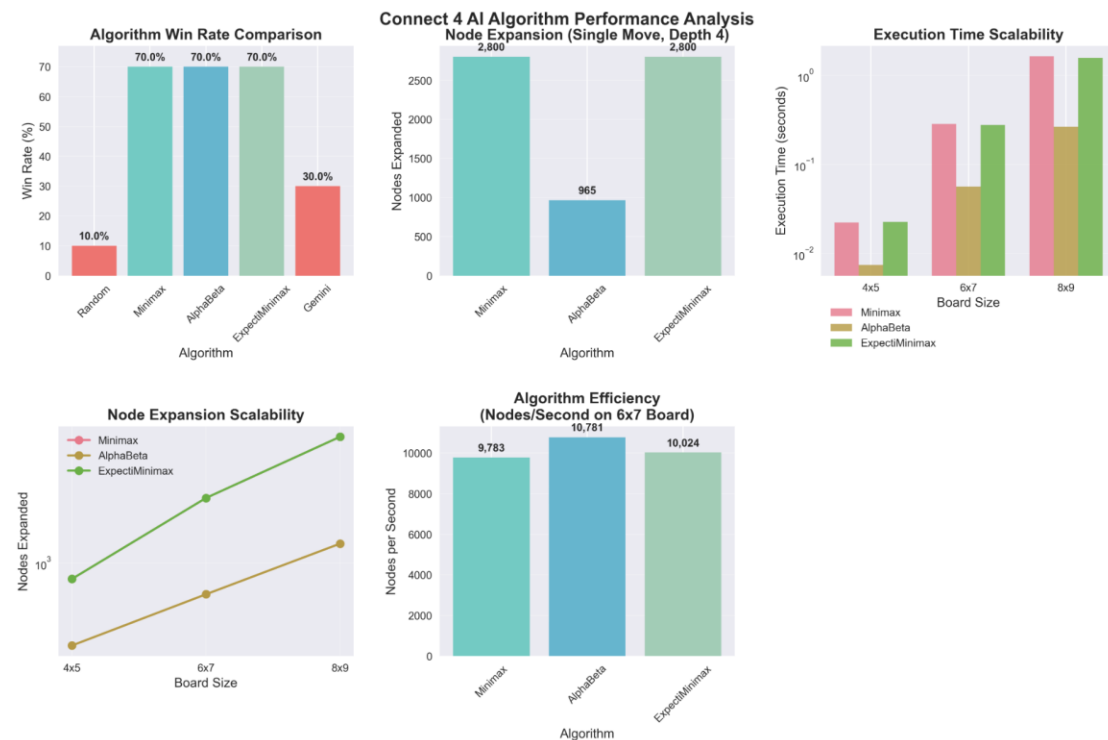
Random Connect 4 Results (No Gemini): Trials = 3



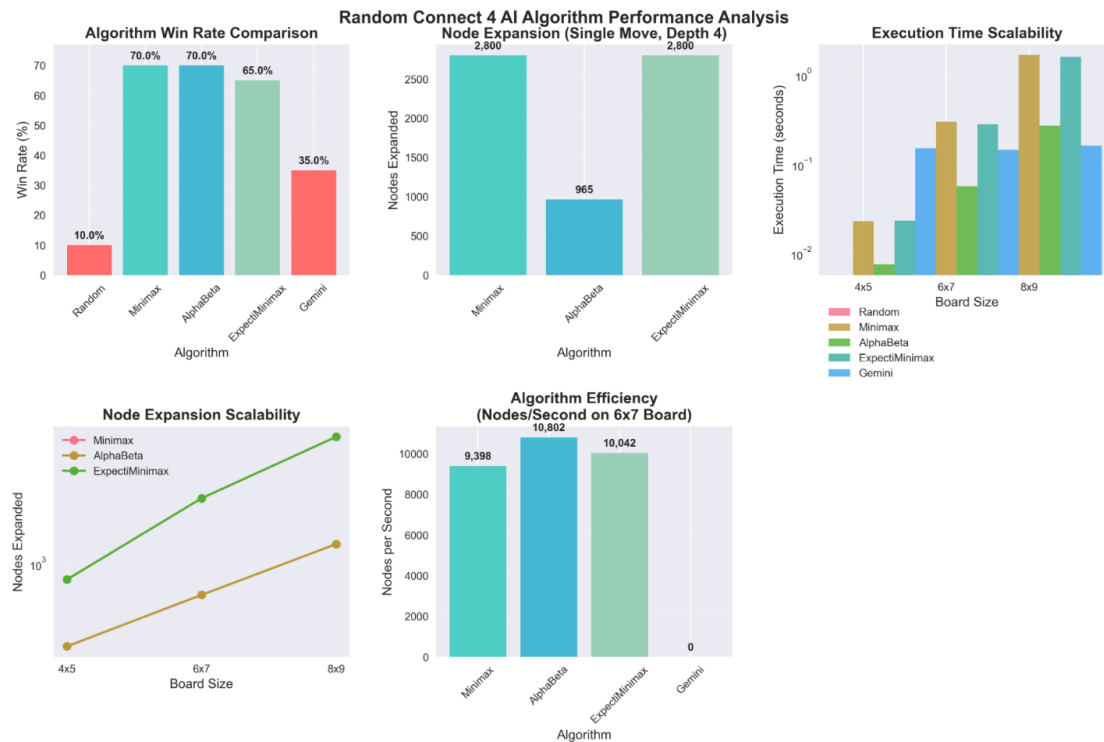
Random Connect 4 Results (No Gemini): Trials = 100



Connect 4 Agent vs Agent with Gemini: Trials = 2



Random Connect 4 Results with Gemini: Trials = 2



- Error 429 occurred; couldn't test algorithm efficiency

Section 4: Sources

Science Buddies. (2024, February 7). Simple Explanation of the Minimax

Algorithm with Alpha-Beta Pruning with Connect 4 [Video]. YouTube.

<https://www.youtube.com/watch?v=DV5d31z1xTI&t=197s>

Dartmouth College. (2016). Lab Assignment 6: Connect Four.

Cs.dartmouth.edu.

<https://www.cs.dartmouth.edu/~scot/cs10/lab/lab6/lab6.html#:~:text=checkers%20in%20an%20unblocked%204,place%20where%20the%20ofourth%20checker>